PROPOSED SOLUTION · OPTION 2

Event Processor — Direct-to-HPOS Ingestion

A thin, stateless consumer that reads events from Kafka and writes them as-is into the HPOS Raw_Events bucket, then commits the offset. No scheduler, no in-memory windows, no feedback topic. Reading raw events and generating Parquet is owned by the Aggregation Layer.

STATUS Proposed SCOPE Ingestion & raw persistence LOAD 2 billion events / month

1. Architecture Overview

Each Event Processor instance reads events from the Kafka topic and writes them **as-is** into the HPOS `Raw_Events` bucket, then commits the offset. There is **no scheduler, no in-memory windowing, and no feedback topic**. This keeps instances fully stateless and shrinks the crash blast radius to a single poll batch.

Figure 1 — Direct-to-HPOS ingestion. Consumers write raw events as-is; the offset commits only after a successful write. Parquet generation moves to the Aggregation Layer.

2. Why remove the scheduler & buffer

- No JVM heap windows → no GC pressure or OOM risk under load spikes.
- Crash blast radius shrinks from a whole 5-minute window to a single poll batch.
- Fully stateless instances — trivial to scale and restart.
- Simpler code path: consume → write → commit. Fewer moving parts to fail.

3. Kafka Topic Handling

- **Single consumer group** (`group.id = event-processor`) — each partition is owned by exactly one consumer, so every event is processed once. A second group would receive its own full copy.
- **HA via rebalance** — partitions reassign automatically across the group on instance loss.
- **Even partition spread** is set at the producer — publish with a unique `eventId` (UUID) as the partition key so load hashes evenly.
- **Durability & buffering** — replication factor 3 keeps data safe; retention longer than any HPOS outage lets the topic safely hold the backlog.

```
topic              = wf.recipient.billable.events
partitions         = 12
replication.factor = 3
retention.ms       = 604800000   # 7 days
cleanup.policy     = delete
producer.acks      = all
partition.key      = eventId (UUID)
```

4. Offset Handling

- The window/batch **manual-commit machinery is gone** — no checkpoint table, no state machine.
- A lightweight **commit-after-write** remains: PUT to HPOS succeeds → commit the offset. That single ordering is what keeps zero loss.
- On replay, the deterministic object key (the `eventId`) makes a re-write idempotent — no duplicates at rest.

5. Write to HPOS Raw_Events

- Events written raw (JSON) — no transform, no Parquet at this stage.
- **One object per event**, keyed by its `eventId` — a deterministic name that makes replays idempotent.

```
key  raw_events/date=2026-07-09/hour=18/p=3/{eventId}.json
```

Optional — if HPOS supports it. To cut PUT volume and object count at high load, flush a **micro-batch** (one object per `poll()`, e.g. up to 1,000 events) via batched / multipart upload — still stateless, no scheduler. Kept as a better option where feasible; per-event write remains the default.

6. HPOS Resiliency & Extended Outages

Resilient writes — when to pause

- Each write is retried with **exponential backoff** (1s → 2s → 4s → 8s, capped ~30s).
- A **circuit breaker** counts consecutive failures. On breach (e.g. 5 in a row or backoff cap hit) → `consumer.pause()` on all assigned partitions.
- While paused the consumer keeps calling `poll()` (fetches nothing) so heartbeats continue and **no rebalance** is triggered.
- A background probe pings HPOS; on success (half-open → closed) → `consumer.resume()`.

If HPOS is down for a long time

- **Kafka is the durable buffer** — unconsumed records stay in the topic; nothing is lost while paused.
- Size topic **retention beyond worst-case outage** (e.g. `retention.ms = 7d`) with disk headroom. Loss only if outage > retention.
- No app-memory backpressure risk — the backlog lives in Kafka's log, **not JVM heap** (key gain over Option 1).
- Alert on `circuit-open`, consumer lag, and paused-duration so ops act before retention is at risk.

7. Parallelism & Capacity

Proposed load is **2 billion events / month**. Max parallelism equals the partition count; scale by adding partitions, not consumers per partition.

772/sec	~3,000/sec	12	3–4
Average throughput	Planned peak (~4×)	Kafka partitions	Instances (scale to 12)

≈ 250/sec average and ~1,000/sec peak per partition across 12 — comfortable for Kafka and a batched writer. Scale to 24 partitions for extra headroom.

8. Moved to the Aggregation Layer

- Reading raw events from HPOS and **generating Parquet files**.
- Event-time bucketing, late-arrival grace windows, and compaction.
- Business & mandatory-parameter validation and billing rules (unchanged).

9. Failure Scenarios

Scenario	Behaviour
Consumer crash before write	Offset uncommitted → Kafka replays the poll batch. Nothing lost.
Crash after write, before commit	Replay re-writes the same eventId key — idempotent, no duplicate.
HPOS unavailable	Retry with backoff; if it persists, pause the consumer. No retry queue needed.
Kafka rebalance	Partitions reassign automatically; stateless consumers resume cleanly.

10. Option 1 vs Option 2

	Option 1 · Scheduler	Option 2 · Direct-to-HPOS
Scheduler	Yes (every 5 min)	No
In-memory buffer	5-min heap windows	None (stateless)
Parquet generation	In Event Processor	In Aggregation Layer
Crash blast radius	Whole active window	One poll batch
Offset commit	After parquet upload	After object write
Feedback topic	Yes	No (deferred)
Small-file risk	Low (10k batched)	Higher — mitigate via micro-batch
Complexity	Higher	Lower